

AbangCebu AI — Protected Route & Middleware Test Scenarios

Document Version: 1.0.0
Status: Approved
Jira Ticket References: [SCRUM-71](#), [SCRUM-113](#) (Route Guards & Middleware Security Test Cases) **Sprint:** Sprint 1 (Foundations & Core Infrastructure)
Author: Karla Hiyas (QA Engineer)
Reviewed & Audited by: Hermar Centillas (Lead / Scrum Master)

Related Specifications: - [Auth Session Lifecycle](#) (SCRUM-57) - [RBAC Permission Matrix](#) (SCRUM-63) - [Auth Error Handling & Edge Cases](#) (SCRUM-60) - [RBAC Test Cases](#) (SCRUM-70) - [Auth Test Plan](#) (SCRUM-69) - [RLS Policies](#) (SCRUM-55)

1. Executive Summary

This document specifies the complete test scenario catalog for the Next.js Edge Middleware (`middleware.ts`) route protection layer in AbangCebu AI. The middleware is the first line of defense in the platform's three-tier defense-in-depth security architecture:

- 1. **Layer 1 — Next.js Edge Middleware** (this document): Fast edge-level session presence checking and role-prefix route matching with HTTP 302 redirects.
- 2. **Layer 2 — Server Route Handlers / Server Actions:** Authoritative JWT verification, database profile role check, and record ownership validation.
- 3. **Layer 3 — Supabase PostgreSQL RLS Policies:** Database-level hard barrier preventing unauthorized reads/writes regardless of application-layer bugs.

These test scenarios validate Layer 1 behavior: unauthenticated redirect enforcement, role-based route guarding, session cookie renewal bypass, public asset passthrough, `?next=` return URL handling, and suspended account interception.

Test Framework: Playwright (E2E) + Vitest (Unit/Integration)
Total Scenarios: 35 test cases across 7 test suites

2. Route Protection Reference Map

The following route protection rules are sourced directly from [docs/security/rbac-matrix.md](#) Section 3:

Route Pattern	Protection Level	Unauthenticated Visitor	Authenticated (Wrong Role)
<code>/</code> , <code>/map</code> , <code>/search</code>	Public	Allow	Allow
<code>/listings/[id]</code>	Public (Approved only)	Allow	Allow
<code>/login</code> , <code>/register</code>	Guest only	Allow	Redirect to role home
<code>/account/*</code>	Authenticated	Redirect 302 → <code>/login?next=<path></code>	Allow (scoped to own profile)
<code>/renter/*</code>	<code>role = 'renter'</code>	Redirect 302 → <code>/login?next=<path></code>	Render HTTP 403 Forbidden
<code>/landlord/*</code>	<code>role = 'landlord'</code>	Redirect 302 → <code>/login?next=<path></code>	Render HTTP 403 Forbidden
<code>/admin/*</code>	<code>role = 'admin'</code>	Redirect 302 → <code>/login?next=<path></code>	Render HTTP 403 Forbidden

Middleware Bypass Paths (No Session Evaluation)

Path Pattern	Reason
/_next/static/*	Next.js static assets (JS, CSS, fonts)
/_next/image/*	Next.js image optimization endpoint
/favicon.ico, /robots.txt, /sitemap.xml	Standard web root files
/api/health	Infrastructure health check probe
/api/auth/callback	Supabase OAuth callback endpoint

3. Test Suites

Suite 1: Public Route Access (TC-MW-001 to TC-MW-005)

These scenarios verify that public routes are accessible without authentication and that authenticated users can also access them freely.

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-001	Guest accesses landing page /	No session cookie	1. Navigate to /	HTTP 200, landing page renders with MapLibre map canvas	P0
TC-MW-002	Guest accesses /map	No session cookie	1. Navigate to /map	HTTP 200, map view renders without login prompt	P0
TC-MW-003	Guest accesses /search?q=cit-u	No session cookie	1. Navigate to /search?q=cit-u	HTTP 200, search results page renders with query preserved	P1
TC-MW-004	Guest accesses /listings/abc123 (approved)	No session cookie, listing status = approved	1. Navigate to /listings/abc123	HTTP 200, listing detail renders (landlord contact hidden per RBAC)	P0
TC-MW-005	Authenticated renter accesses /map	Valid renter session	1. Navigate to /map	HTTP 200, map renders with authenticated navbar (avatar, logout)	P1

Suite 2: Unauthenticated Redirect Enforcement (TC-MW-006 to TC-MW-012)

These scenarios verify that unauthenticated users are redirected to /login with the proper ?next= return URL parameter when attempting to access protected routes.

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-006	Guest attempts /renter/dashboard	No session cookie	1. Navigate to /renter/dashboard	HTTP 302 redirect to /login?next=%2Frenter%2Fdashboard	P0
TC-MW-007	Guest attempts /landlord/listings/new	No session cookie	1. Navigate to /landlord/listings/new	HTTP 302 redirect to /login?next=%2Flandlord%2Flistings%2Fnew	P0

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-008	Guest attempts / admin/users	No session cookie	1. Navigate to /admin/users	HTTP 302 redirect to /login?next=%2Fadmin%2Fusers	P0
TC-MW-009	Guest attempts / account/settings	No session cookie	1. Navigate to /account/settings	HTTP 302 redirect to /login?next=%2Faccount%2Fsettings	P0
TC-MW-010	Guest attempts / renter/saved with query params	No session cookie	1. Navigate to /renter/saved?sort=price&near=citu	HTTP 302 redirect to /login?next=%2Frenter%2Fsaved%3Fsort%3Dprice%26near%3Dcit-u — query string preserved in return URL	P1
TC-MW-011	Guest attempts / landlord/inquiries/inbox	No session cookie	1. Navigate to /landlord/inquiries/inbox	HTTP 302 redirect to /login?next=%2Flandlord%2Finquiries%2Finbox	P1
TC-MW-012	Guest attempts / admin/listings/pending	No session cookie	1. Navigate to /admin/listings/pending	HTTP 302 redirect to /login?next=%2Fadmin%2Flistings%2Fpending	P1

Suite 3: Role-Based Route Guard Enforcement (TC-MW-013 to TC-MW-020)

These scenarios verify that authenticated users are blocked from accessing routes belonging to different roles, rendering an HTTP 403 Forbidden page.

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-013	Renter attempts / landlord/listings/new	Valid renter session (role = 'renter')	1. Navigate to /landlord/listings/new	HTTP 403 Forbidden page with "Access Denied" message and link to /renter/dashboard	P0
TC-MW-014	Renter attempts / admin/users	Valid renter session	1. Navigate to /admin/users	HTTP 403 Forbidden page	P0
TC-MW-015	Landlord attempts / renter/saved	Valid landlord session (role = 'landlord')	1. Navigate to /renter/saved	HTTP 403 Forbidden page with link to /landlord/dashboard	P0
TC-MW-016	Landlord attempts / admin/listings/pending	Valid landlord session	1. Navigate to /admin/listings/pending	HTTP 403 Forbidden page	P0
TC-MW-017	Admin attempts / renter/dashboard	Valid admin session (role = 'admin')	1. Navigate to /renter/dashboard	HTTP 403 Forbidden page with link to /admin/dashboard	P1
TC-MW-018	Admin attempts / landlord/listings/new	Valid admin session	1. Navigate to /landlord/listings/new	HTTP 403 Forbidden page	P1

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-019	Renter attempts <code>/ landlord/units/unit-456/edit</code>	Valid renter session	1. Navigate to <code>/ landlord/units/unit-456/edit</code>	HTTP 403 Forbidden — deep nested landlord route blocked	P1
TC-MW-020	Landlord attempts <code>/ admin/moderation/ flagged</code>	Valid landlord session	1. Navigate to <code>/ admin/moderation/ flagged</code>	HTTP 403 Forbidden — admin moderation route blocked	P1

Suite 4: Authenticated User on Guest-Only Routes (TC-MW-021 to TC-MW-024)

These scenarios verify that already-authenticated users are redirected away from login/register pages to their role-appropriate dashboard.

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-021	Renter navigates to <code>/ login</code>	Valid renter session	1. Navigate to <code>/login</code>	HTTP 302 redirect to <code>/renter/dashboard</code> (or <code>/search</code>)	P0
TC-MW-022	Landlord navigates to <code>/ register</code>	Valid landlord session	1. Navigate to <code>/ register</code>	HTTP 302 redirect to <code>/ landlord/dashboard</code>	P0
TC-MW-023	Admin navigates to <code>/ login</code>	Valid admin session	1. Navigate to <code>/login</code>	HTTP 302 redirect to <code>/admin/dashboard</code>	P1
TC-MW-024	Renter navigates to <code>/ login?next=/renter/saved</code>	Valid renter session	1. Navigate to <code>/login?next=/renter/saved</code>	HTTP 302 redirect to <code>/renter/saved</code> (honors <code>?next=</code> parameter)	P1

Suite 5: Session Cookie Renewal & Edge Middleware Behavior (TC-MW-025 to TC-MW-029)

These scenarios verify the middleware's silent background session refresh behavior and cookie handling, as specified in the [Auth Session Lifecycle](#).

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-025	Active session with TTL > 300s (pass-through)	Valid session cookie, access token TTL = 1200s remaining	1. Navigate to <code>/ renter/dashboard</code>	HTTP 200, no <code>Set-Cookie</code> mutation, request forwarded directly	P0
TC-MW-026	Active session with TTL < 300s (silent refresh)	Valid session cookie, access token TTL = 120s remaining	1. Navigate to <code>/ renter/dashboard</code>	HTTP 200, response includes <code>Set-Cookie</code> header with refreshed <code>sb-<ref>-auth-token</code> cookies	P0
TC-MW-027	Expired session cookie (full redirect)	Expired/invalid session cookie, refresh token also expired	1. Navigate to <code>/ landlord/listings</code>	HTTP 302 redirect to <code>/login?next=%2Flandlord%2Flistings</code> — treated as unauthenticated	P0
TC-MW-028	Corrupted/tampered session cookie	Malformed cookie value (not valid JWT)	1. Navigate to <code>/ admin/users</code>	HTTP 302 redirect to <code>/login</code> — middleware gracefully rejects tampered token	P0
TC-MW-029	Missing cookie but valid <code>Authorization</code> header	No cookie, but <code>Authorization: Bearer <token></code> in request	1. Send fetch request to <code>/api/listings</code> with Bearer token	API route processes the request (middleware does not interfere with API auth headers for server-to-server calls)	P2

Suite 6: Static Asset & API Bypass Paths (TC-MW-030 to TC-MW-033)

These scenarios verify that the middleware correctly bypasses session evaluation for static assets, health checks, and OAuth callback endpoints.

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-030	Request to <code>/_next/static/chunks/main.js</code>	No session cookie	1. Request <code>/_next/static/chunks/main.js</code>	HTTP 200 (or 304), JS bundle served without redirect	P0
TC-MW-031	Request to <code>/_next/image?url=...&w=640&q=75</code>	No session cookie	1. Request image optimization endpoint	HTTP 200, optimized image served without redirect	P1
TC-MW-032	Request to <code>/api/health</code>	No session cookie	1. GET <code>/api/health</code>	HTTP 200 with <code>{"status": "ok"}</code> response — health probe not redirected to login	P0
TC-MW-033	Request to <code>/api/auth/callback?code=...</code>	OAuth redirect from Supabase/Google	1. GET <code>/api/auth/callback?code=abc123</code>	HTTP 200 (or 302 to dashboard), callback processed without middleware blocking	P0

Suite 7: Suspended Account Interception (TC-MW-034 to TC-MW-035)

These scenarios verify that the middleware detects suspended accounts (`profiles.is_suspended = true`) and prevents access even with a valid session token, as specified in the [Auth Session Lifecycle](#).

ID	Scenario	Preconditions	Steps	Expected Result	Priority
TC-MW-034	Suspended renter attempts protected route	Valid session cookie, but <code>profiles.is_suspended = true</code> set by admin	1. Navigate to <code>/renter/dashboard</code>	Middleware checks suspension status, clears session cookie, redirects to <code>/login?reason=suspended</code> with banner: "Your account has been suspended. Contact support."	P0
TC-MW-035	Suspended landlord attempts <code>/landlord/listings</code>	Valid session cookie, <code>profiles.is_suspended = true</code>	1. Navigate to <code>/landlord/listings</code>	Session invalidated, redirect to <code>/login?reason=suspended</code>	P0

4. Test Data Requirements

4.1 Test User Accounts

Test User	Role	Email	Session State	Purpose
<code>test-renter-01</code>	renter	<code>renter01@test.abangcebu.ph</code>	Valid, active	Standard renter route access
<code>test-landlord-01</code>	landlord	<code>landlord01@test.abangcebu.ph</code>	Valid, active	Standard landlord route access
<code>test-admin-01</code>	admin	<code>admin01@test.abangcebu.ph</code>	Valid, active	Admin route access
<code>test-suspended-renter</code>	renter	<code>suspended.renter@test.abangcebu.ph</code>	Valid, <code>is_suspended = true</code>	Suspension interception

Test User	Role	Email	Session State	Purpose
test-suspended-landlord	landlord	suspended.landlord@test.abangcebu.ph	Valid, is_suspended = true	Suspension interception
test-expired-session	renter	expired@test.abangcebu.ph	Expired token	Expired session redirect

4.2 Test Listings

Listing ID	Status	Owner	Purpose
listing-approved-001	approved	test-landlord-01	Public listing access test
listing-pending-001	pending	test-landlord-01	Non-public listing visibility test

5. Automation Implementation Notes

5.1 Playwright E2E Test Structure

```
// tests/e2e/middleware/route-protection.spec.ts

import { test, expect } from '@playwright/test';

test.describe('Suite 1: Public Route Access', () => {
  test('TC-MW-001: Guest accesses landing page', async ({ page }) => {
    const response = await page.goto('/');
    expect(response?.status()).toBe(200);
    await expect(page.locator('[data-testid="map-canvas"]')).toBeVisible();
  });

  test('TC-MW-004: Guest accesses approved listing', async ({ page }) => {
    const response = await page.goto('/listings/listing-approved-001');
    expect(response?.status()).toBe(200);
    // Landlord contact info should be hidden for guests
    await expect(page.locator('[data-testid="landlord-phone"]')).not.toBeVisible();
  });
});

test.describe('Suite 2: Unauthenticated Redirect Enforcement', () => {
  test('TC-MW-006: Guest redirected from /renter/dashboard', async ({ page }) => {
    await page.goto('/renter/dashboard');
    expect(page.url()).toContain('/login');
    expect(page.url()).toContain('next=%2Frenter%2Fdashboard');
  });

  test('TC-MW-008: Guest redirected from /admin/users', async ({ page }) => {
    await page.goto('/admin/users');
    expect(page.url()).toContain('/login');
    expect(page.url()).toContain('next=%2Fadmin%2Fusers');
  });
});

test.describe('Suite 3: Role-Based Route Guard', () => {
  test('TC-MW-013: Renter blocked from /landlord/listings/new', async ({ page }) => {
    // Login as renter first (use auth fixture)
    await loginAsRenter(page);
    const response = await page.goto('/landlord/listings/new');
    expect(response?.status()).toBe(403);
    await expect(page.locator('text=Access Denied')).toBeVisible();
  });
});
```

5.2 Vitest Unit Test Structure (Middleware Isolation)

```
// tests/unit/middleware.test.ts

import { describe, it, expect, vi } from 'vitest';
import { middleware } from '@middleware';
import { NextRequest } from 'next/server';

describe('Middleware: Static Asset Bypass', () => {
  it('TC-MW-030: should not evaluate session for static assets', async () => {
    const req = new NextRequest('http://localhost:3000/_next/static/chunks/main.js');
    const res = await middleware(req);
    // Middleware should return NextResponse.next() without redirect
    expect(res.status).not.toBe(302);
  });

  it('TC-MW-032: should bypass /api/health', async () => {
    const req = new NextRequest('http://localhost:3000/api/health');
    const res = await middleware(req);
    expect(res.status).not.toBe(302);
  });
});

describe('Middleware: Unauthenticated Redirect', () => {
  it('TC-MW-006: should redirect guest from /renter/dashboard', async () => {
    const req = new NextRequest('http://localhost:3000/renter/dashboard');
    // No session cookie set
    const res = await middleware(req);
    expect(res.status).toBe(302);
    expect(res.headers.get('Location')).toContain('/login');
    expect(res.headers.get('Location')).toContain('next=%2Frenter%2Fdashboard');
  });
});
```

6. Coverage Matrix

Test Suite	Count	P0	P1	P2	Automation
Suite 1: Public Route Access	5	3	2	0	Playwright E2E
Suite 2: Unauthenticated Redirect	7	4	3	0	Playwright E2E + Vitest
Suite 3: Role-Based Route Guard	8	4	4	0	Playwright E2E
Suite 4: Guest-Only Route Redirect	4	2	2	0	Playwright E2E
Suite 5: Session Cookie Renewal	5	3	0	2	Vitest (mocked Supabase)
Suite 6: Static Asset Bypass	4	2	1	1	Vitest (unit)
Suite 7: Suspended Account	2	2	0	0	Playwright E2E
Total	35	20	12	3	—

7. Traceability to Architecture Specifications

Middleware Behavior	Source Specification	Section
Route prefix matching (/renter/*, /landlord/*, /admin/*)	RBAC Permission Matrix (SCRUM-63)	§3 Route Protection Rules

Middleware Behavior	Source Specification	Section
?next=<path> return URL parameter	Auth Session Lifecycle (SCRUM-57)	§1.2 Source Document Mapping
Silent token refresh (TTL < 300s)	Auth Session Lifecycle (SCRUM-57)	§1.3 Scope diagram
Suspended account interception (is_suspended)	Auth Session Lifecycle (SCRUM-57)	§1.2 Source Document Mapping
Guest-only route redirect (logged-in user on /login)	RBAC Permission Matrix (SCRUM-63)	§3 Route Protection Rules
Static asset bypass (/_next/static/*)	Next.js Edge Middleware convention	Built-in framework behavior
Defense-in-depth 3-tier model	RBAC Permission Matrix (SCRUM-63)	§3 Defense-in-Depth Enforcement Layers
Error codes (AUTH_ACCOUNT_SUSPENDED)	Auth Error Handling (SCRUM-60)	§2.1 Error Response Schema

8. Definition of Done

- [x] 35 test scenarios documented across 7 suites
- [x] Route protection rules traced to RBAC matrix (SCRUM-63)
- [x] Session renewal behavior traced to Auth Session Lifecycle (SCRUM-57)
- [x] Error codes traced to Auth Error Handling (SCRUM-60)
- [x] Playwright E2E and Vitest unit test code samples provided
- [x] Test data requirements specified (6 test users, 2 test listings)
- [x] Coverage matrix with priority classification (P0/P1/P2)
- [x] Document registered in docs/README.md

9. Sign-Off

Role	Name	Status
Author	Karla Hiyas (QA Engineer)	✅ Authored
Reviewer	Hermar Centillas (Lead / Scrum Master)	✅ Approved